<Bolsa Futuro Digital>

<aula 27>

Turma de FrontEnd - IF UFRGS

Short Circuiting

O JavaScript é preguiçoso: ele para de avaliar uma expressão assim que souber o resultado.

Com ||, se o primeiro valor for verdadeiro, o segundo nem é avaliado.

Com &&, se o primeiro valor for falso, o segundo nem é avaliado.

Isso não é só true/false — qualquer valor pode ser retornado!

```
// Valores falsy no JavaScript:
// false, 0, '', null, undefined, NaN

// Valores truthy: tudo o mais!
// true, 1, 'texto', [], {}, ...

// || retorna o primeiro valor TRUTHY:
console.log(false || 'Visitante')   // 'Visitante'
console.log('Ana'  || 'Visitante')   // 'Ana'

// && retorna o primeiro valor FALSY
// (ou o último valor, se todos forem truthy):
console.log(false && 'clicado')     // false
console.log(true  && 'clicado')     // 'clicado'
```

Short Circuiting

```
// Retorna o PRIMEIRO valor truthy
// ou o último, se todos forem falsy

let a = 0   || 'zero'   // 'zero'
let b = ''  || 'vazio'  // 'vazio'
let c = null|| 'nulo'   // 'nulo'
let d = 'Ana'|| 'João'   // 'Ana'

// Sem short circuit (com if/else):
let nome = ''
let exibir
if (nome) {
  exibir = nome
} else {
  exibir = 'Visitante'
}

// Com || (muito mais curto!):
let exibir2 = nome || 'Visitante'
// 'Visitante'
```

```
// Uso prático: valor padrão

let usuario = {
  nome:  'Ana',
  idade: null
}

// exibe o valor ou um padrão:
let nome  = usuario.nome  || 'Anônimo'
let idade = usuario.idade || 'Não informada'

console.log(nome)   // 'Ana'
console.log(idade)  // 'Não informada'
```

Short Circuiting

```
// Retorna o PRIMEIRO valor falsy
// ou o último, se todos forem truthy

let a = false && 'texto'   // false
let b = 0     && 'texto'   // 0
let c = true  && 'texto'   // 'texto'
let d = true  && 42        // 42
let e = true  && true      // true

// Sem short circuit:
let logado = true
if (logado) {
  console.log('Bem-vindo!')
}

// Com &&:
logado && console.log('Bem-vindo!')
// 'Bem-vindo!'
```

```
// Uso prático: ação condicional

let admin = true
let div = document.getElementById('painel')

// Mostra painel só se for admin:
admin && (div.style.display = 'block')

// Combinando && e ||:
let logado = false

let status = (logado && 'Online') || 'Offline'
console.log(status)  // 'Offline'

logado = true
status = (logado && 'Online') || 'Offline'
console.log(status)  // 'Online'
```

Short Circuiting

```
// Problema com || e o número 0:
let pontos = 0
let exibir = pontos || 'Sem pontos'
console.log(exibir)
// 'Sem pontos' ← errado! pontos = 0

// 0 é falsy → || ativa o lado direito
// mas 0 é um valor VÁLIDO aqui!

// O mesmo com string vazia:
let descricao = ''
let texto = descricao || 'Sem descrição'
console.log(texto)
// 'Sem descrição' ← '' pode ser intencional!
```

```
// ?? só ativa se for null ou undefined
// Ignora 0, '', false, NaN

let pontos = 0
let exibir = pontos ?? 'Sem pontos'
console.log(exibir)  // 0 ✓ correto!

let descricao = ''
let texto = descricao ?? 'Sem descrição'
console.log(texto)   // '' ✓ correto!

// ?? ativa só com null/undefined:
let a = null      ?? 'padrão'  // 'padrão'
let b = undefined ?? 'padrão'  // 'padrão'
let c = 0         ?? 'padrão'  // 0
let d = ''        ?? 'padrão'  // ''
let e = false     ?? 'padrão'  // false
```

Short Circuiting

|| → use para valor padrão quando o valor pode ser qualquer coisa falsy (null, undefined, '', false...)

?? → use para valor padrão apenas quando o valor pode ser null ou undefined (e você quer preservar 0, '', false)

&& → use para executar algo condicionalmente, só se a condição for verdadeira

```
// Exibir mensagem de boas-vindas:
let usuario = { nome: 'Ana', admin: true }

let saudacao = 'Olá, ' + (usuario.nome || 'Visitante')
console.log(saudacao)  // 'Olá, Ana'

let badge = usuario.admin && '[ADMIN]'
console.log(badge)     // '[ADMIN]'

// Placar de jogo (0 é válido → use ??):
let placar = 0
let exibir = placar ?? 'Partida não iniciada'
console.log(exibir)    // 0

// Status de conexão:
let loogado = false
let status = (logado && 'Online') || 'Offline'
console.log(status)    // 'Offline'
```

.forEach( )

forEach() percorre um array e transforma cada item usando uma callback. O array original **é modificado.**

É como mandar um array "passar por uma fábrica" — cada item entra, é transformado, e sai modificado do outro lado.

```
// forEach() => itera sobre todos os elementos
// de um array e aplica um função (callback)

let valores = [1, 2, 3, 4, 5]

function dobrar(elemento, indice, array) {
    // x2 elemento na posição nº indice do array
    array[indice] = elemento * 2
}

function display(elemento) {
    console.log(elemento)
}

// mostra na tela cada elemento
valores.forEach(display)
// multiplica por 2 cada elemento
valores.forEach(dobrar)
// mostra na tela cada elemento
valores.forEach(display)
```

.map( )

map() percorre um array e transforma cada item, retornando um novo array com os resultados. O array original **nunca é modificado.**

É muito parecido com o forEach(), mas um novo array é criado, sem modificar o original.

A função que você passa recebe o item atual como argumento e deve retornar o **novo valor.**

```
// Sintaxe:
let novoArray1 = array.map(function(item) {
  return /* novo valor */
})

// Exemplo simples:
let numeross = [1, 2, 3, 4]

let dobrados = numeros.map(function(n) {
  return n * 2
})

console.log(numeros)   // [1, 2, 3, 4]  ← original intacto!
console.log(dobrados)  // [2, 4, 6, 8]  ← novo array

// map() SEMPRE retorna um array
// com o MESMO número de itens.
```

.filter( )

filter() percorre um array e mantém apenas os itens para os quais a função retorna true.

O resultado é um **novo array**, possivelmente menor — o original não é modificado.

A função que você passa recebe o item atual e deve retornar true (manter) ou false (descartar).

```
// .filter cria um array novo
//   filtrando os elementos

const idades = [16, 17, 18, 18, 19, 20, 60]

function maiorIdade (num) {
    // retorna true ou false
    return num >= 18
}

function menorIdade (num) {
    // retorna true ou false
    return num < 18
}

// Apenas idades maiores que 18
const adultos = idades.filter(maiorIdade)
// Apenas idades menores que 18
const jovens = idades.filter(menorIdade)

console.log(adultos)
console.log(jovens)
```

Exercícios

Exercício 1: Crie um objeto representando um usuário com as propriedades nome, idade, premium (booleano true) e bio. Em seguida, crie quatro variáveis: uma que exibe o nome ou 'Anônimo' se ele for falsy; uma que exibe a idade ou 'Não informada'; uma que exibe '[PREMIUM]' apenas se o usuário for premium; e uma que exibe a bio ou 'Sem bio', sem substituir uma bio vazia '' por engano. Exiba as quatro variáveis no console.

Exercício 2: Crie um array com três produtos, cada um com as propriedades nome e preco. Use forEach para aplicar um desconto de 20% no preço de cada produto, modificando o array diretamente. Depois use um segundo forEach para exibir cada produto no console no formato "Camiseta: R$ 64".

</aula 27>